

# Do Professional Developers Benefit from Design Pattern Documentation? A Replication in the Context of Source Code Comprehension

Carmine Gravino<sup>1</sup>, Michele Risi<sup>1</sup>,  
Giuseppe Scanniello<sup>2</sup>, and Genoveffa Tortora<sup>1</sup>

<sup>1</sup> Facoltà di Scienze MM.FF.NN., Università Degli Studi di Salerno, Italy  
{gravino,mrisi,tortora}@unisa.it

<sup>2</sup> Dipartimento di Matematica e Informatica, Università della Basilicata, Italy  
giuseppe.scanniello@unibas.it

**Abstract.** We present the results of a differentiated replication conducted with professional developers to assess whether the presence and the kind of documentation for the solutions or instances of design patterns affect source code comprehension. The participants were divided into three groups and asked to comprehend a chunk of the JHot-Draw source code. Depending on the group, each participant was or not provided with the graphical and textual representations of the design pattern instances implemented within that source code. In the case of graphically documented instances, we used UML class diagrams, while textually documented instances are reported as comment in the source code. The results revealed that participants provided with the documentation of the instances achieved a significantly better comprehension than the participants with source code alone. The effect of the kind of documentation is not statistically significant.

**Keywords:** Design Patterns, Controlled Experiment, Maintenance, Replications, Software Models, Source Code Comprehension.

## 1 Introduction

Software maintenance is essential in the evolution of software systems and represents one of the most expensive, time consuming, and challenging phases of the whole development process. Maintenance starts after the delivery of the first version of the system and lasts much longer than the initial development process [5], [32]. As shown in the survey by Erlikh [10], the cost needed to perform maintenance operations ranges from 85% to 90% of the total cost of a software project. Whatever is the maintenance operation, the greater part of the cost and effort are due to the comprehension of source code [20]. In particular, Pfleeger and Atlee [23] estimated that up to 60% of software maintenance is spent on comprehension. There are several reasons that make comprehension even more costly and complex, namely the size of a subject software and the available documentation [28].

The availability of software documentation and software models should provide a better support to comprehend source code, so reducing the needed effort and positively affecting the efficiency with which developers perform maintenance operations [2]. For example, Gamma *et al.* [11] assert that developers would benefit from the documentation of design patterns to comprehend source code, so easing its modification. Although there are a number of empirical investigations on design patterns (e.g., [6], [7], [15], [19], [22], [24], [29], [30]), only few evaluations have been conducted on the practical benefits of explicitly reporting design pattern instances<sup>1</sup> in the comprehension of source code [12], [25]. Furthermore, there are no empirical investigations using professional software developers as the participants.

In this paper, we present the results of a differentiated replication<sup>2</sup> conducted with 25 professional software developers to assess whether the presence and the kind of design pattern instances affect source code comprehension. The participants have been working for software companies of the contact network of the authors' research groups. This network was created from research projects and also included companies that: (i) host students from the universities of Basilicata and Salerno for external internships or (ii) employ people who took a Master or a Bachelor degree at these universities. The participants were divided into three groups and were asked to perform a comprehension task on the source code of JHotDraw. Depending on the group, the participants were provided with source code added or not with design pattern instances either graphically or textually documented. To explicitly and graphically show these instances, we used UML class diagrams [21], while textually documented instances are reported as comment in the source code according to a template.

The work presented here is based on [12] and with respect to it the following new contributions are provided: (1) a differentiated replication with professional developers; (2) a different analysis on the effect of graphically and textually documented design pattern instances; and (3) a deeper discussion on the achieved results and on the possible future directions for this research.

The paper is organized as follows. In Section 2, we highlight the previously conducted controlled experiments and how design pattern instances are documented in these experiments and in the replication presented here. In Section 3, we show the design of this replication, while in Section 4 we show and discuss the results achieved. Related work, remarks, and future work conclude the paper.

## 2 Documenting Design Pattern Instances

In the design of buildings and towns a design pattern describes a problem which occurs over and over again in our environment, and then describes the core of the

---

<sup>1</sup> A design pattern includes a name, an intent, a problem, its solution, some example, and so on [11]. In the paper, we focus on the solutions and we will refer to them as design pattern instances.

<sup>2</sup> In this kind of replication, variations in essential aspects (e.g., different kinds of participants) of the original experimental conditions are introduced [3].

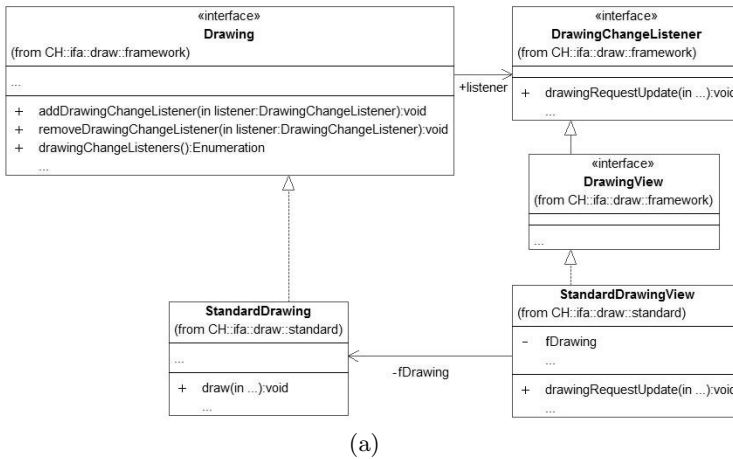
solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice [1]. This definition also holds for design patterns in object-oriented software [11]. The core of both kinds of design patterns is a solution to a problem in a given context. In object-oriented software development a solution is named design pattern instance [13].

There is no a single standard format for documenting design patterns and their instances. Rather, a variety of different formats have been proposed and many of them are based on the UML (e.g., [11], [14]). However, only few studies have been conducted to assess the support provided by explicitly reporting design pattern instances in the execution of maintenance operations and in the comprehension of source code [12], [25]. In [12], for example, we presented a controlled experiment and a replication to assess the benefit of documenting instances with respect to not documenting instances at all. In the first experiment, we considered graphically documented instances by using UML class diagrams, while we considered textually documented ones in the replication.

Each graphical representation of design pattern instances showed a superset of the information provided by the corresponding textual representation. For example, in both the representations the roles each class played within the pattern instances were indicated, while in the textually documented instances the relations among classes (both abstract and concrete) and interfaces were not shown. Figure 1(a) shows an example of graphically documented instance of the Observer design pattern [11] within the source code of JHotDraw. Figure 1(b) shows how the same instance is explicitly reported within the source code as comment. The graphical instance shows more information and then should improve the comprehension of source code. For example, from the class diagram, we can understand that when a drawing (e.g, a container of figures) is changed all the views are updated. More experienced professional software developers could find unnecessary the further information that graphically documented instance provides, because they can directly deduce it from the name of the design pattern and the role of each class and interface.

## 2.1 Previously Conducted Experiments

The first experiment (UNIBAS in the following) was conducted with 17 Master Students in Computer Science at the University of Basilicata. The participants to the second experiment (a differentiated replication, UNISA in the following) were 24 Master Students in Computer Science at the University of Salerno. The main differences between UNIBAS and UNISA concerned: (i) the participants involved and (ii) the design used. The participants had a level of experience comparable, but they came from different universities located in different regions. All the involved participants had basic software engineering knowledge. In particular, they knew the basics of requirements engineering, high- and low-level design of object-oriented software systems based on the UML, software development, and software maintenance. They had, however, a limited experience in developing and maintaining nontrivial software systems.



```

1: /*
2:  * @(#)DrawingChangeListener.java 5.1
3:  *
4:  * Observer Pattern
5:  * Subject: Drawing
6:  * ConcreteSubject: StandardDrawing
7:  * Observer: DrawingChangeListener
8:  * ConcreteObserver: StandardDrawingView
9:  */
10:
11: package CH.ifa.draw.framework;
12:
13: public interface DrawingChangeListener
14:     extends EventListener {
15:
16:     /**
17:      * ...
18:      */
19:     public void drawingRequestUpdate(...);
20: }

```

(b)

**Fig. 1.** A sample of instance for the Observer design pattern: graphically documented (a) and textually documented (b)

Regarding the differences in the design, in UNIBAS the instances were graphically documented by UML class diagrams. For UNISA, the instances were textually documented within the source code in terms of comment. In both the experiments, the participants were asked to comprehend a chunk of JHotDraw v5.1. A single factor experimental design was used in both the experiments. The main factor was represented by the kind of documentation used to explicitly report the instances. This factor was denoted as *Method* and could assume two values: DP (Design Pattern instance documentation) and SC (Source Code alone). For UNIBAS, DP assumed the meaning of graphically documented instances (from here on, GD), while in the replication textually documented (TD). For SC, the source code did not contain any reference to the included instances.

To assess the effect of *Method* on a comprehension task, we considered three dependent variables that measure: (i) source code comprehension (*Comprehension*);

(ii) the time to comprehend source code (*Effort*); and (iii) comprehension task efficiency (*Efficiency*). All the three variables are ratio scale measures.

**Results.** The results of the data analysis on both the experiments provided evidence that participants achieved better Comprehension values when they used the documentation of design pattern instances as complementary information to the source code. The results also indicated that the capability of the participants to correctly recognize design pattern instances impacted more than the type of representation employed to document them. Furthermore, the participants in both the experiments indicated that they trusted the explicitly reported design pattern instances and found them useful.

As far as Efficiency is concerned, we observed that the participants were more efficiently supported in the execution of comprehension tasks when source code was added with the documentation of the design pattern instances. For Effort, the participants to UNIBAS significantly spent less time when using design pattern instances with respect to source code alone. We did not observe any significant difference for UNISA.

### 3 The Replication with Professionals

The replication was carried out by following the recommendations provided in [17], [31]. The presentation of the replication is based on the guidelines suggested in [16]. For replication purposes, we made available on the Web<sup>3</sup> an experimental package, the raw data, and a technical report with some analyses not reported here for space reasons.

#### 3.1 Goal

Applying the Goal Question Metric (GQM) paradigm [4], the goal of the replication can be defined as: *Analyse* the use of graphical and textual documentation for design pattern instances *for the purpose* of evaluating them *with respect to* the source code comprehension *from the point of view* of project manager, *in the context of* professional software developers.

#### 3.2 Context Selection

We conducted the experiment with 25 Italian software professionals. For each company, we organized a laboratory session. This was the only possible strategy because it is practically impossible to conduct a single experimental session with professionals from different companies. All the laboratory sessions were carried out under controlled conditions to avoid biasing the results, the experiment supervisors were the same in each session.

---

<sup>3</sup> [www.dmi.unisa.it/people/risi/www/DesignPatternInstancesComprehension/](http://www.dmi.unisa.it/people/risi/www/DesignPatternInstancesComprehension/)

Before the controlled experiment, each professional was asked to fill in a pre-questionnaire. This questionnaire was sent and returned by email. The information gathered was used to classify the participants as junior (with working experience from 1 to 3 years) and senior (with an experience more than 3 years) professional software developers. The junior developers were 10, while 15 were classified as senior. The participants stated that their experience on design pattern development was from *low* to *medium*.

### 3.3 Selection of the Variables

The dependent variables are: Comprehension, Effort, and Efficiency. To compute the values of Comprehension, we asked each participant to answer a comprehension questionnaire composed of 14 open questions. To quantify the quality of the answers provided and then the source code comprehension achieved, we used an approach based on the information retrieval theory [27]. Therefore, we defined: (1)  $A_{s,i}$  as the set of string items provided as answer to the question  $i$  by the participant  $s$ ; (2)  $C_i$  as the correct set of items expected for the question  $i$  (i.e., the oracle<sup>4</sup>). For each answer, we can compute:

$$precision_{s,i} = \frac{|A_{s,i} \cap C_i|}{|A_{s,i}|} \quad recall_{s,i} = \frac{|A_{s,i} \cap C_i|}{|C_i|}$$

Precision (i.e., the fraction of items in the answer that are correct) and recall (i.e., the fraction of correct items in the answer) measure the correctness of the answers to a given question and the completeness of the answers, respectively. To get a balance between correctness and completeness, we used a standard aggregated measure based on the combination of precision and recall:

$$F-Measure_{s,i} = \frac{2 \cdot precision_{s,i} \cdot recall_{s,i}}{precision_{s,i} + recall_{s,i}}$$

For each participant, the Comprehension value is computed by performing the overall average of the F-Measure values of all the questions. Comprehension assumes values in the interval  $[0, 1]$ . A value close to 1 means that a participant got a very good comprehension of the source code since he/she answered very well to the questions of the comprehension questionnaire. Conversely, a value close to 0 means that a participant obtained a very bad comprehension.

To determine Effort, we used the time (expressed in minutes) to accomplish the task, which was directly recorded by each participant, while Efficiency was computed dividing Comprehension by Effort. Efficiency is a derived measure that we considered to get a deeper understanding of the contribution provided by the documentation of design pattern instances in the comprehension of source

---

<sup>4</sup> The names of the classes and methods in the oracle might be different between TD and SC as well as between TD and GD. This is because we removed any possible reference to the design pattern instances from the comment and from the identifiers when the participants used SC and GD.

code. The higher the value of Efficiency, the more efficiently the participant is supported in the accomplishment of the task.

Method is the only independent variable used. It is a nominal variable and assumes values in {SC, TD, GD}. We also grouped the professionals (of TD and GD) into participants, who correctly or incorrectly identified the needed design pattern instances to answer the questions of the comprehension questionnaire: DPCI (i.e., Design Patterns Correctly Identified) and DPnCI (i.e., Design Patterns not Correctly Identified). The data analysis was conducted considering the Comprehension values for the participants in these groups. We performed this further analysis to understand whether different design pattern instances affect source code comprehension. It was possible because we asked the participants to indicate the instances exploited to answer each question.

### 3.4 Hypotheses Formulation and Experiment Design

We have defined and investigated the following null hypotheses:

**Hn0\_D\_X.** The participants who used D design pattern instances (where D can be GD or TD) did not achieve significantly better results in terms of X (where X can be Effort, Comprehension, or Efficiency) than the participants who used source code alone (SC).

**Hn1\_X.** There was not a significant difference with respect to X when participants used GD or TD.

Hn0\_D\_X is one-tailed because we expected a positive effect of explicitly reporting design pattern instances on the selected dependent variables. Hn1\_X is two-tailed because we could not postulate any effect of GD or TD on these variables. The goal of the statistical analysis is to reject the defined null hypotheses and to accept the alternative ones (i.e., Ha0\_D\_X and Ha1\_X), which can be easily derived (e.g., Ha1\_X: There was a significant difference with respect to X when participants used GD or TD).

We used the one factor with three treatments design [31]. The participant working experience (i.e., the amount of years as professional developers) was the blocking factor. Then, we equally distributed junior and senior experienced professionals among the three groups: GD, TD, and SC. We assigned 9 participants (4 juniors and 5 seniors) to GD and 8 (3 juniors and 5 seniors) to TD and SC, respectively. The use of a different experiment design (such as the within-participant counterbalanced design) with non-trivial experimental objects (as in this experiment) may bias the results introducing a factor difficult to be controlled, i.e., the mental fatigue.

### 3.5 Experimental Tasks

We asked the participants to perform the following tasks:

**Comprehension Task.** The participants were asked to fill in the comprehension questionnaire, whose questions were divided into three groups to let

|                                                                                                                                     |                                          |                                       |                                           |                                      |
|-------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|---------------------------------------|-------------------------------------------|--------------------------------------|
| <b>Q2.</b> Indicating the class/es and the method/s in charge of creating, drawing, and updating the instances of the class Figure? |                                          |                                       |                                           |                                      |
| How much do you trust your answer <sup>+</sup> ?                                                                                    |                                          |                                       |                                           |                                      |
| <input type="checkbox"/> Unsure                                                                                                     | <input type="checkbox"/> Not sure enough | <input type="checkbox"/> Sure Enough  | <input type="checkbox"/> Sure             | <input type="checkbox"/> Very Sure   |
| How do you assess the question <sup>+</sup> ?                                                                                       |                                          |                                       |                                           |                                      |
| <input type="checkbox"/> Very difficult                                                                                             | <input type="checkbox"/> Difficult       | <input type="checkbox"/> On average   | <input type="checkbox"/> Simple           | <input type="checkbox"/> Very Simple |
| What is the source of information used to answer the question <sup>+</sup> ?                                                        |                                          |                                       |                                           |                                      |
| <input type="checkbox"/> Previous Knowledge (PK)                                                                                    |                                          | <input type="checkbox"/> Internet (I) | <input type="checkbox"/> Source Code (SC) |                                      |
| <sup>+</sup> Mark only one answer                                                                                                   |                                          |                                       |                                           |                                      |

**Fig. 2.** A question example from the comprehension questionnaire

participants take a break if needed when passing from a group of questions to the next one. This choice was taken for reducing fatigue effect biases. We defined the questions to assess several aspects related to the comprehension of the source code. All the questions (except Q11) were formulated using a similar form/schema. Figure 2 shows a sample question for SC.

We also collected data on the source of information the participants used to answer each question. In particular, we asked the participants who accomplished the task with source code added with documented design pattern instances (i.e., GD and TD) to specify for each question whether the answer was derived using: (DPI) design pattern instances, (PK) previous knowledge, (I) Internet, or source code (SC). If the participants specified DPI, they were also asked to indicate the instances used. The participants who accomplished the task using the source code alone chose among: previous knowledge, Internet, and source code. This was the only difference introduced in the comprehension questionnaires used in the three treatments. Whatever was the treatment, we asked the participant to indicate also the confidence level (e.g., *Sure*) and the degree of complexity (e.g., *Difficult*) for each question answered (see Figure 2). The analysis on this further information is not reported for space constraint, but it is available in the technical report.

The question in Figure 2 expected as the correct answer the following set of items: *CreationTool*, *ArrayFigure*, *StandardDrawingView*, *createFigure()*, *draw()*, and *drawingRequestUpdate()*. The correct answer could be derived by the following instances of design patterns: Prototype, Composite, and Observer (see Figure 1). In particular, the Prototype instance was useful because it was in charge of managing the creation of a template figure, while the Composite drew each base element of an object Figure. The Observer instance was in charge of managing the paint and/or the repaint of an object Figure. If a participant provided *CreationTool*, *ArrayFigure*, and *drawingChangeListener()* as the answer, the value for Comprehension is 0.44. It results from 0.66 and 0.33 as the precision and recall values, respectively. In fact, the number of correct items provided is 2 (*CreationTool* and *ArrayFigure*), while 3 is the total number of items provided and 6 is the number of correct items expected.



**Post-experiment Task.** We asked the participants to fill in a post-experiment survey questionnaire. The goal of this questionnaire was to obtain feedback about the participants' perceptions of the experiment execution. For space reasons the results of the survey are not reported in the paper. Details can be found in the technical report.

### 3.6 Experimental Procedure

The participants first attended an introductory lesson in which the supervisors presented detailed instructions on the experiment. The supervisors highlighted the goal of the experiment without providing details on the experimental hypotheses. No time limit to perform the task was imposed. We organized individual experimental sessions for professional developers working in the same business unit. The participants were not allowed to communicate each other.

To perform the comprehension task, the participants were provided with laptops having the same hardware configuration (i.e., equipped with a 1.5 GHz Intel Centrino with 1.5 GB of RAM, a 60GB Hard Disk and Windows XP Professional SP3 as operating system). To surf source code, we installed on each laptop a general purpose and well known text editor (i.e., UltraEdit<sup>5</sup>). We also provided the participant with an Internet connection to be used while performing the comprehension task.

We asked the participants to use the following experimental procedure for each group of questions within the comprehension questionnaire: *(i)* specifying name and start-time; *(ii)* answering the questions using the source code (without executing it) and the explicitly reported design pattern instances if present; and *(iii)* marking the end-time. We did not suggest any approach to comprehend source code. We only discouraged to read all the code.

We provided the participants with a paper copy of the following experimental material: *(i)* the comprehension questionnaire and *(ii)* a post-experiment survey questionnaire. The participants in GD were also provided with the source code (without any references to the design pattern instances) and the paper copy of a document where each design pattern instance was graphically reported (see Figure 1(a)). The participants that used TD were provided with source code that included the references to the design pattern instances in the comment (see Figure 1(b)). For SC, the participants were provided with source code without any kind of documentation to the instances implemented.

### 3.7 Analysis Procedure

To perform the data analysis, we carried out the following steps and used the R environment<sup>6</sup> for statistical computing:

1. We undertook the descriptive statistics of the measures of the dependent variables, i.e., Effort, Comprehension, and Efficiency (see Section 4.1).

<sup>5</sup> [www.ultraedit.com](http://www.ultraedit.com)

<sup>6</sup> [www.r-project.org](http://www.r-project.org)

2. To test the null hypotheses, we adopted non-parametric tests due to the sample size and mostly the non-normality of the data. In particular, we used the Mann-Whitney test [9] due to the design of the experiments (only unpaired analyses were possible) and to its robustness [31] (see Section 4.2). In all the statistical tests, we decided (as custom) to accept a probability of 5% of committing Type-I-error [31]. The chosen statistical test allows the presence of a significant difference between independent groups to be verified, but it does not provide any information about this difference [18]. Therefore, we used the Cohen's  $d$  [8] effect size to obtain the standardized difference between two groups that can be considered negligible for  $|d| < 0.2$ , small for  $0.2 \leq |d| < 0.5$ , medium for  $0.5 \leq |d| < 0.8$ , and large for  $|d| \geq 0.8$ . We also analyzed the statistical power for each test performed. Statistical power is the probability that the test will reject a null hypothesis when it is actually false (i.e., the probability of not committing a Type II error, or making a false negative decision). The highest value is 1, while 0 is the lowest. The higher the statistical power value, the higher is the probability to reject a null hypothesis when it is actually false.

### 3.8 Differences and Similarities

The experience gained in the previously executed experiments [12] suggested some variations in the experiment presented here. The variations have been introduced to mitigate as many threats to validity as possible and to improve the material and the data analysis:

**Participants.** They are professional developers and are more experienced than the participants to UNIBAS and UNISA. This variation allowed reducing external validity threats.

**Experiment Design.** We used the one factor with three treatments design. The participants were divided into three groups. The control group was the group of participants in SC. Differently, we have here two treatment groups: GD and TD. As for UNIBAS and UNISA, the independent variable is Method (i.e., the main factor), which is a nominal variable that assumes three possible values: SC, TD, and GD.

**Group Composition.** We used the information gathered in a pre-questionnaire to equally distribute high and low experienced professionals among the three groups. The professional experience is the blocked factor for the experiment.

**Data Analysis.** Bearing in mind the new adopted design, we were able to better analyze the effect of the documentation type on source code comprehension.

**Training Session.** The professionals did not carried out a training session on tasks similar to the one used in the experiment. Two were the reasons: (1) they had an adequate experience in performing maintenance operations on source code implemented by others; (2) time and logistic constraints did not make possible the execution of a training session (the use of professionals might cause this kind of concern).

**Experimental Procedure.** We allowed the participants to find information on the Web useful to accomplish that task. Professional developers usually exploit this medium as support for their daily work activities.

**Comprehension Questionnaire.** We removed mistakes and some sources of possible confusion.

We preserved some design choices in the replication presented here:

**Dependent Variables.** They are well known and widely employed in the Empirical Software Engineering community (e.g., [26]). These variables well summarize the aspects we were interested in investigating. Another by-product of this choice was in the evaluation of source code comprehension that could be computed in a repeatable manner, so reducing construct validity threats.

**Experimental Object.** We used a chunk (i.e., vertical slice) of JHotDraw v5.1 that included: (i) a nontrivial number of design pattern instances and (ii) well-known and widely adopted design patterns. In the selection process, we have also taken into account a trade-off between the complexity of the implemented functionality and the effort to comprehend it (about 3 hours for low experienced participants). To mitigate external validity threats, we tried as much as possible to define a realistic comprehension task.

We translated the comments from English into Italian to avoid biasing the results because different participants may have different familiarity with English. Further, we removed any possible reference to the design pattern instances from the comment and from the identifiers (e.g. *CompositeFigure* was named as *ArrayFigure*) when the participants performed the comprehension task with the source code alone and the graphically documented instances. The source code was constituted of 1326 Lines of Code, 26 Classes, and 823 Lines of Comments. One of the authors manually detected the design pattern instances in source code. To this end, he also used the documentation of JHotDraw and the public dataset PMARt<sup>7</sup>. The following instances of design patterns were present in the source code used: State, Adapter, Strategy, Decorator, Composite, Observer, Command, Template Method, and Prototype. For the State design pattern were two instances. These instances are graphically represented (as much as possible) as in [11] and textually represented as shown in Section 2. We used JHotDraw because it is intentionally designed to have very clear implementations of well-known design patterns. Therefore, it can be considered a good experimental object.

## 4 Results

### 4.1 Descriptive Statistics and Exploratory Analysis

Table 1 shows some descriptive statistics (i.e., median, mean, and standard deviation) of Effort, Comprehension, and Efficiency grouped by Method. These

---

<sup>7</sup> [www.ptidej.net/downloads/pmart/](http://www.ptidej.net/downloads/pmart/)

**Table 1.** Descriptive statistics for GD, TD, and SC

| Dependent Variable | GD    |        |          | TD    |        |          | SC    |        |          |
|--------------------|-------|--------|----------|-------|--------|----------|-------|--------|----------|
|                    | Mean  | Median | St. Dev. | Mean  | Median | St. Dev. | Mean  | Median | St. Dev. |
| Effort             | 132   | 142.2  | 31.17    | 139   | 136.4  | 37.00    | 151   | 147.4  | 42.81    |
| Comprehension      | 50.91 | 51.08  | 10.32    | 53.43 | 53.49  | 7.26     | 40.56 | 39.97  | 9.63     |
| Efficiency         | 0.37  | 0.37   | 0.11     | 0.37  | 0.42   | 0.14     | 0.29  | 0.31   | 0.14     |

**Table 2.** Descriptive statistics for GD grouped by DPCI and DPnCI

| Dependent Variable | DPCI |        |          | DPnCI |        |          |
|--------------------|------|--------|----------|-------|--------|----------|
|                    | Mean | Median | St. Dev. | Mean  | Median | St. Dev. |
| Effort             | 9    | 10.36  | 3.93     | 8.00  | 9.13   | 3.67     |
| Comprehension      | 80   | 73     | 27.70    | 47    | 39.13  | 36.88    |
| Efficiency         | 8.17 | 8.24   | 4.60     | 4.4   | 5.28   | 6.18     |

statistics show that the participants using source code alone (SC) spent on average more time (151 minutes) than the participants using documented design pattern instances (132 and 139 minutes for GD and TD, respectively). On average the participants who used GD and TD achieved a better comprehension of source code (50.91 and 53.43, respectively) than those who used SC (40.56). We achieved similar results for Efficiency.

Table 2 shows descriptive statistics (i.e., median, mean, and standard deviation) of Effort, Comprehension, and Efficiency for GD grouping observations by DPCI and DPnCI. Similarly, Table 3 reports descriptive statistics for TD. These descriptive statistics suggest that the participants who correctly recognized the design pattern instances (both in TD and GD), to answer a given question, achieved on average better Comprehension and Efficiency values than the participants who did not correctly recognized them.

4.2 Hypotheses Testing

The results of the Mann-Whitney test are summarized in Table 4, together with the Cohens’ d effect size and the statistical power values. The results show that Hn0\_GD\_Comprehension and Hn0\_TD\_Comprehension can be rejected (p-values are 0.033 and 0.009, respectively) with a large effect size and high statistical power. Thus, the participants who used the documentation of design pattern instances significantly better comprehended source code than those provided with source code alone. Hn0\_D\_Effort and Hn0\_D\_Efficiency cannot be rejected.

**Table 3.** Descriptive statistics for TD grouped by DPCI and DPnCI

| Dependent Variable | DPCI |        |          | DPnCI |        |          |
|--------------------|------|--------|----------|-------|--------|----------|
|                    | Mean | Median | St. Dev. | Mean  | Median | St. Dev. |
| Effort             | 10   | 9.61   | 3.41     | 10    | 9.97   | 4.93     |
| Comprehension      | 73   | 67.06  | 30.88    | 45    | 38.50  | 36.13    |
| Efficiency         | 7.69 | 8.46   | 5.91     | 3.94  | 5.76   | 8.08     |

Table 4. Results for Hn0\_D\_X

| Documentation | Hypothesis     | Influence (p-value) | Effect Size         | Statistical Power |
|---------------|----------------|---------------------|---------------------|-------------------|
| GD            | └Effort        | No (0.337)          | -0.137 (negligible) | 0.075             |
|               | └Comprehension | Yes (0.033)         | 1.113 (large)       | 0.966             |
|               | └Efficiency    | No (0.135)          | 0.527 (medium)      | 0.287             |
| TD            | └Effort        | No (0.282)          | -0.273 (small)      | 0.115             |
|               | └Comprehension | Yes (0.009)         | 1.586 (large)       | 0.875             |
|               | └Efficiency    | No (0.056)          | 0.806 (large)       | 0.354             |

Table 5. Results for Hn1\_X

| Hypothesis     | Influence (p-value) | Effect Size        | Statistical Power |
|----------------|---------------------|--------------------|-------------------|
| └Effort        | No (0.810)          | 0.170 (negligible) | 0.045             |
| └Comprehension | No (0.665)          | -0.271 (small)     | 0.065             |
| └Efficiency    | No (0.664)          | -0.375 (small)     | 0.075             |

Table 5 shows the data analysis results for the null hypotheses Hn1\_X. In particular, the results of the Mann-Whitney test indicated that the null hypotheses cannot be rejected. Therefore, for all the dependent variables the difference between the participants who used graphically documented and textually documented design pattern instances is not statistical significant.

4.3 Further Analyses - Analysis by Question

The DPCI participants in GD achieved significant better results in terms of Comprehension than the DPnCI participants on the questions Q1 (p-value = 0.043) and Q5 (p-value 0.048). This result suggested that the design patterns that better supported the participants in the execution of comprehension tasks were: Prototype, Composite, Observer, and Template Method. Regarding TD, the DPCI participants achieved significantly better results in terms of Comprehension on the question Q3 (p-value = 0.032). This indicated that only Composite and Observer design patterns better supported the participants in the source code comprehension. This result is interesting from the researcher’s point of view because it seems that the interaction among the instances of different design patterns affects source code comprehension. Further and special conceived investigations are, however, needed because our primary goal here was: to assess whether the presence and the kind of documentation for design pattern instances affect source code comprehension. In this further analysis, we did not considered Q11 because it was formulated differently from the others.

4.4 Discussion

The results of this replication have largely confirmed those achieved in the previous experiments [12]. The software professionals achieved significant better Comprehension values when they received the documentation of the design pattern instances as a complementary information to comprehend source code.

In particular, the mean improvement achieved with the design pattern instances graphically documented was 25.5%, while it was 31.9% for those textually documented. The participants who used textually documented design pattern instances obtained on average slightly better results in terms of Comprehension with respect to the participants who exploited graphically documented instances. A plausible justification for this result is that professionals were more comfortable with source code and the information provided in the comment (i.e., explicitly reported instances) was more than adequate to comprehend the code.

The time to perform the comprehension task did not increase with respect to the use of the source code alone. This result could be considered as unexpected because more documents/information to read and interpret could need more effort to execute the task. This should be even more evident for design pattern instances that were graphically documented. Then, these results suggest that the additional information provided by the documented design pattern instances reduced the effort to analyze the source code.

For GD, the descriptive statistics reported in Table 2 indicate that the average Comprehension value achieved by the participants when they correctly identified the instances to answer the question is 70.2% greater than the average value obtained when the instances were not correctly recognized. For TD, this difference is 62.2%. This finding is interesting from both the researcher and the project manager points of view. In fact, it seems relevant to help developers in recognizing pattern instances more than the kind of documentation used.

Regarding the source of information, we observed that the participants employing GD largely indicated as first source of information the design patterns. Differently, the participants employing TD indicated as first source of information the source code, while design pattern instances were classified as the second one. This slight difference in the results achieved on GD and TD could be due to the fact that the design pattern instances are documented in the source code in the latter case. Then, the participants considered the documented instances as an integral part of the code. Although this difference, the results show that the participants trusted the design pattern instances explicitly reported. It is also worth noting that the Internet was almost never used: 8 participants (6 used TD and 2 GD) stated that they on average used the Internet on 2 out of 14 questions. Therefore, the Internet was not considered a relevant source of information.

#### 4.5 Threats to Validity

*Conclusion validity* concerns issues that affect the ability of drawing a correct conclusion. In our study, we used proper statistical tests. In particular, a non-parametric test (i.e., Mann-Whitney test for unpaired analyses) was used to statistically reject the null hypotheses.

*Internal validity* threats are mitigated by the design of the experiment. Each group of participants worked only on one task, with or without the design patterns instances. Fatigue is another possible threat for internal validity. We mitigate the fatigue effect allowing the participants to take a break. Another possible

threat concerns the use of the Internet to exchange information. We prevented that monitoring the participants, while performing the task.

*Construct validity* may be influenced by the metrics used and social threats. The exploited metrics are widely used with purposes similarly to ours [26]. Regarding Comprehension, one of the authors not involved in the definition of the task built the questionnaire. A further threat could be related to the modification of the identifiers in the code.

*External validity* concerns the generalization of the results. Possible threats are related to the complexity of the comprehension task and the choice of participants. Regarding the first point, we selected a part of an open software system large enough to be considered not excessively easy. As for the participants, they are Italian professional junior/senior software developers. Moreover Southern and Central Italy are over-represented with respect to Northern Italy.

## 5 Related Work and Conclusion

Only few studies have been conducted to assess the support that design pattern instances provide in the execution of maintenance tasks and in the comprehension of source code [12], [25]. Prechelt *et al.* [25] studied whether design pattern instances explicitly and textually documented in the source code (through comment) improve the maintainers' performance in performing comprehension tasks with respect to a well-commented program without explicit reference to design patterns. The study involved 74 German graduate students and 22 USA undergraduate students, who performed maintenance operations on Java and C++ code, respectively. The data analysis revealed that maintenance tasks supported by explicitly documented design pattern instances were completed faster or with fewer errors. The most remarkable difference with our work is that we additionally analyze the effect of pattern instances graphically documented. Furthermore, we used professionals and the used experimental object is larger and more complex (20149 LOCs including comments with respect to 360 and 560).

The results achieved in our experiment and those achieved in [12] and [25] give strength to the usefulness of exploiting explicitly documented design pattern instances in the execution of comprehension tasks. Therefore, it seems worthily to document design pattern instances. This result, however, opens a managerial dilemma: Are the additional effort and cost, due to create and maintain the documentation of design pattern instances, adequately paid back by an improved comprehension of source code? Indeed, from a manager point of view, the adoption of graphically and textually documentation, as means to represent design pattern instances, should take into account the costs it will introduce. Furthermore, what is the less expensive method for representing instances? These points represent future directions for our work.

Another remarkable result of our experiment is: design pattern based development can increase the source code comprehension only in case the design pattern instances are correctly recognized in the source code. This opens an interesting future direction for the research. In particular, it would be worth investigating: (i) the issues that led to certain patterns being better comprehended and

recognized than others and (ii) new methods for representing design pattern instances, so easing their recognition. It will be also worth investigating whether the source code comprehension improves when graphically documented design pattern instances are added with sequence diagrams.

**Acknowledgments.** We thank the participants to the experiment and Paolo Mondelli for his support.

## References

1. Alexander, C., Ishikawa, S., Silverstein, M., Jacobson, M., Fiksdahl-King, I., Angel, S.: A Pattern Language - Towns, Buildings, Construction. Oxford University Press (1977)
2. Arisholm, E., Briand, L.C., Hove, S.E., Labiche, Y.: The impact of UML documentation on software maintenance: An experimental evaluation. *IEEE Trans. Softw. Eng.* 32(6), 365–381 (2006)
3. Basili, V., Shull, F., Lanubile, F.: Building knowledge through families of experiments. *IEEE Trans. Softw. Eng.* 25(4), 456–473 (1999)
4. Basili, V.R., Rombach, H.D.: The TAME project: Towards improvement-oriented software environments. *IEEE Trans. Software Eng.* 14(6), 758–773 (1988)
5. Bennett, K.H., Rajlich, V.T.: Software maintenance and evolution: a roadmap. In: *Procs. of the Conference on the Future of Software Engineering, ICSE 2000*, pp. 73–87. ACM, New York (2000)
6. Bieman, J., Straw, G., Wang, H., Munger, P., Alexander, R.: Design patterns and change proneness: an examination of five evolving systems. In: *Procs. of Software Metrics Symposium*, pp. 40–49. IEEE CS (2003)
7. Cepeda Porras, G., Guéhéneuc, Y.-G.: An empirical study on the efficiency of different design pattern representations in UML class diagrams. *Empirical Softw. Eng.* 15(5), 493–522 (2010)
8. Cohen, J.: *Statistical power analysis for the behavioral sciences*, 2nd edn. Lawrence Earlbaum Associates, Hillsdale (1988)
9. Conover, W.J.: *Practical Nonparametric Statistics*, 3rd edn. Wiley (1998)
10. Erlikh, L.: Leveraging legacy system dollars for e-business. *IT Professional* 2, 17–23 (2000)
11. Gamma, E., Helm, R., Johnson, R., Vlissides, J.: *Design Patterns: Elements of Reusable Object Oriented Software*. Addison-Wesley (1995)
12. Gravino, C., Risi, M., Scanniello, G., Tortora, G.: Does the documentation of design pattern instances impact on source code comprehension? Results from two controlled experiments. In: *Procs. of the Working Conference on Reverse Engineering*, pp. 67–76. IEEE CS (2011)
13. Guéhéneuc, Y.-G., Antoniol, G.: Demima: A multilayered approach for design pattern identification. *IEEE Trans. Softw. Eng.* 34(5), 667–684 (2008)
14. Heer, J., Agrawala, M.: Software design patterns for information visualization. *IEEE Trans. Visualization & Comp. Graphics (Proc. InfoVis)* 12, 853–860 (2006)
15. Jeanmart, S., Guéhéneuc, Y.-G., Sahraoui, H., Habra, N.: Impact of the Visitor Pattern on program comprehension and maintenance. In: *Procs. of the Symposium on Empirical Software Engineering and Measurement*, pp. 69–78. IEEE CS (2009)



16. Jedlitschka, A., Ciolkowski, M., Pfahl, D.: Reporting Experiments in Software Engineering. In: Shull, F., Singer, J., Sjöberg, D. (eds.) *Guide to Advanced Empirical Software Engineering*, pp. 201–228. Springer, London (2008)
17. Juristo, N., Moreno, A.: *Basics of Software Engineering Experimentation*. Kluwer Academic Publishers (2001)
18. Kampenes, V., Dyba, T., Hannay, J., Sjöberg, I.: A systematic review of effect size in software engineering experiments. *Information and Software Technology* 49(11–12), 1073–1086
19. Khomh, F., Guéhéneuc, Y.-G.: Do design patterns impact software quality positively? In: *Procs. of Conference on Software Engineering and Maintenance*, pp. 274–278 (2008)
20. Mayrhauser, A.V.: Program comprehension during software maintenance and evolution. *IEEE Computer* 28, 44–55 (1995)
21. OMG. Unified modeling language (UML) specification, version 2.0. Technical report, Object Management Group (July 2005)
22. Penta, M.D., Cerulo, L., Guéhéneuc, Y.-G., Antoniol, G.: An empirical study of the relationships between design pattern roles and class change proneness. In: *Procs. of the International Conference on Software Maintenance*, pp. 217–226. IEEE CS (2008)
23. Pfleeger, S., Atlee, J.: *Software engineering - theory and practice*, 3rd edn. Ellis Horwood (2006)
24. Prechelt, L., Unger, B., Tichy, W.F., Brössler, P., Votta, L.G.: A controlled experiment in maintenance comparing design patterns to simpler solutions. *IEEE Trans. Software Eng.* 27(12), 1134–1144 (2001)
25. Prechelt, L., Unger-Lamprecht, B., Philippsen, M., Tichy, W.: Two controlled experiments assessing the usefulness of design pattern documentation in program maintenance. *IEEE Trans. Softw. Eng.* 28(6), 595–606 (2002)
26. Ricca, F., Penta, M.D., Torchiano, M., Tonella, P., Ceccato, M.: How developers' experience and ability influence web application comprehension tasks supported by uml stereotypes: A series of four experiments. *IEEE Trans. Software Eng.* 36(1), 96–118 (2010)
27. Salton, G., McGill, M.J.: *Introduction to Modern Information Retrieval*. McGraw Hill, New York (1983)
28. Selfridge, P., Waters, R., Chikofsky, E.: Challenges to the field of reverse engineering. In: *Proc. of the Working Conference on Reverse Engineering*. IEEE CS (1993)
29. Vokac, M.: Defect frequency and design patterns: An empirical study of industrial code. *IEEE Trans. Software Eng.* 30(12), 904–917 (2004)
30. Vokác, M., Tichy, W.F., Sjöberg, D.I.K., Arisholm, E., Aldrin, M.: A controlled experiment comparing the maintainability of programs designed with and without design patterns—a replication in a real programming environment. *Emp. Softw. Eng.* 9(3), 149–195 (2004)
31. Wohlin, C., Runeson, P., Höst, M., Ohlsson, M., Regnell, B., Wesslén, A.: *Experimentation in Software Engineering - An Introduction*. Kluwer (2000)
32. Zelkowitz, M.V., Shaw, A.C., Gannon, J.D.: *Principles of software engineering and design*. Prentice-Hall (1979)